Introduction to Artificial Intelligence and Machine Learning (CSN11601)

LAB TASK-3

Name : Aayansh Niranjan Date : 06/10/2025
Reg_No.: 20253004

Problem Statement 1: Array Operations in Daily Life

Scenario:

You are working as a data analyst for a sports team. The coach wants to analyze the players'
scores from recent matches. You are given a list of player scores, and your task is to manage
and analyze the data using NumPy arrays.

Task:

Write a Python program that:

1. Creates a NumPy array with the players' scores.
2. Adds a new score to the array.
3. Inserts a score at a specific position in the array.
4. Removes a particular score from the array.
5. Finds the position of a given score.
6. Sorts the scores in ascending order.
7. Reverses the order of the array to show the top scores first.

Example:

If the scores are `[45, 60, 72, 51, 89]`, demonstrate each operation step by step.

A1. CODE:

```
import numpy as np

scores = np.array([45, 60, 72, 51, 89])
print("1. Original scores:", scores)

new_score = 68
scores_added = np.append(scores, new_score)
print("2. After adding", new_score, ":", scores_added)

insert_score = 75
insert_position = 2
scores_inserted = np.insert(scores_added, insert_position, insert_score)
print(f"3. After inserting {insert_score} at index {insert_position}:", scores_inserted)

remove_score = 51
indices = np.where(scores_inserted == remove_score)[0]
if indices.size > 0:
    scores_removed = np.delete(scores_inserted, indices[0])
    print(f"4. After removing {remove_score} (first occurrence):", scores_removed)
else:
    scores_removed = scores_inserted
    print(f"4. Score {remove_score} not found. No removal. Current:", scores_removed)

find_score = 72
positions = np.where(scores_removed == find_score)[0]
if positions.size > 0:
    print(f"5. Position(s) of score {find_score} (0-based indices):", positions.tolist())
else:
    print(f"5. Score {find_score} not found.")

sorted_scores = np.sort(scores_removed)
print("6. Sorted in ascending order:", sorted_scores)

top_scores = sorted_scores[::-1]
print("7. Reversed (top scores first):", top_scores)
```

OUTPUT:

```
1. Original scores: [45 60 72 51 89]
2. After adding 68 : [45 60 72 51 89 68]
3. After inserting 75 at index 2: [45 60 75 72 51 89 68]
4. After removing 51 (first occurrence): [45 60 75 72 89 68]
5. Position(s) of score 72 (0-based indices): [3]
6. Sorted in ascending order: [45 60 68 72 75 89]
7. Reversed (top scores first): [89 75 72 68 60 45]
```

Problem Statement 2: Smart Calculator for Daily Use

Scenario:

You are building a simple calculator app for your friend who is learning math. The calculator
should help perform basic operations and also handle both real and complex numbers for
more advanced calculations.

Task:

Write a Python program that:
1. Defines separate functions for addition, subtraction, multiplication, and division.
2. Uses the math module to perform operations like finding the ceiling value of a number, exponential value, and greatest common divisor.
3. Uses the cmath module to find the tangent, logarithm base 10, and to check if two complex numbers are approximately equal.
4. Allows the user to input numbers and select which operation to perform.

Example:

If the user inputs two numbers like 5 and 3, the calculator should display results for all four
operations. If complex numbers are given, it should use cmath functions.

A2. CODE:

```
import math
import cmath

def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        return "Error: Division by zero!"
    return a / b

def math_operations(x, y):
    print("\n--- Math Module Operations (for real numbers) ---")
    print("Ceiling value of", x, ":", math.ceil(x))
    print("Exponential value of", x, ":", math.exp(x))
    print("GCD of", int(x), "and", int(y), ":", math.gcd(int(x), int(y)))

def complex_operations(z1, z2):
    print("\n--- CMath Module Operations (for complex numbers) ---")
    print("Tangent of", z1, ":", cmath.tan(z1))
    print("Log base 10 of", z1, ":", cmath.log10(z1))
    print("Are both complex numbers approximately equal?:", cmath.isclose(z1, z2))

print("===== Smart Calculator =====")
choice = input("Do you want to use complex numbers? (y/n): ").lower()

if choice == 'y':
    a = complex(input("Enter first complex number (e.g., 2+3j): "))
    b = complex(input("Enter second complex number (e.g., 1+4j): "))
    print("\n--- Basic Operations ---")
    print("Addition:", add(a, b))
    print("Subtraction:", subtract(a, b))
    print("Multiplication:", multiply(a, b))
    print("Division:", divide(a, b))
    complex_operations(a, b)

else:
    a = float(input("Enter first number: "))
    b = float(input("Enter second number: "))
    print("\n--- Basic Operations ---")
    print("Addition:", add(a, b))
    print("Subtraction:", subtract(a, b))
    print("Multiplication:", multiply(a, b))
    print("Division:", divide(a, b))
    math_operations(a, b)
```

OUTPUT:

```
(base) user@wipro-desktop:~/Desktop/Aayansh_20253004/Lab_Task3$ python p2.py
===== Smart Calculator =====
Do you want to use complex numbers? (y/n): n
Enter first number: 123
Enter second number: 321

--- Basic Operations ---
Addition: 444.0
Subtraction: -198.0
Multiplication: 39483.0
Division: 0.38317757009345793

--- Math Module Operations (for real numbers) ---
Ceiling value of 123.0 : 123
Exponential value of 123.0 : 2.6195173187490626e+53
GCD of 123 and 321 : 3
(base) user@wipro-desktop:~/Desktop/Aayansh_20253004/Lab_Task3$ python p2.py
===== Smart Calculator =====
Do you want to use complex numbers? (y/n): y
Enter first complex number (e.g., 2+3j): 4+2j
Enter second complex number (e.g., 1+4j): 21+3j

--- Basic Operations ---
Addition: (25+5j)
Subtraction: (-17-1j)
Multiplication: (78+54j)
Division: (0.2+0.06666666666666668j)

--- CMath Module Operations (for complex numbers) ---
Tangent of (4+2j) : (0.0364233692474037+1.0046823121902348j)
Log base 10 of (4+2j) : (0.6505149978319906+0.20135959813668655j)
Are both complex numbers approximately equal?: False
```